Informe ejecutivo – CodificaMed

Nov 3, 2026 · @Daniel

Codificación automática CIE-10 de historias clínicas con PLN

Escuela Superior Politécnica de Chimborazo · Facultad de Informática y Electrónica · Escuela de Ingeniería en Sistemas
Materia: Base de Conocimiento · Unidad: Procesamiento de Lenguaje Natural
Estudiante: Daniel Carpio · Docente: Ing. Isaac Torres

1. Problemática y objetivo

CodificaMed es un prototipo que sugiere códigos CIE-10 de diagnóstico a partir de una nota clínica en español, muestra la frase literal que justifica cada código y separa los diagnósticos negados de los confirmados.

Problemática. En la Red Pública Integral de Salud (MSP, IESS) y en las clínicas privadas, cada atención se registra con códigos CIE-10 que sirven para la facturación, la epidemiología y la planificación de recursos. Hoy un codificador lee notas en texto libre, con abreviaturas (“HTA”, “DM2”), errores de tipeo y negaciones (“niega fiebre”, “se descarta neumonía”), y asigna los códigos a mano. El proceso es lento y propenso a errores; un código mal asignado genera glosas, retrasa la facturación y degrada las estadísticas de morbilidad. Un error especialmente costoso es codificar como presente un diagnóstico que la nota niega.

Objetivo. Construir un prototipo que, dada una nota clínica, proponga códigos CIE-10 de diagnóstico, cada uno con:

- la evidencia: fragmento copiado literalmente de la nota;
- el estado: confirmado, negado o en sospecha.

Así el codificador humano valida en lugar de codificar desde cero. Todo código sugerido debe existir en el catálogo CIE-10 y toda evidencia debe ser texto literal de la nota. El sistema debe funcionar con un modelo local (Ollama) y con una API externa (Gemini), cambiando entre ambos solo por configuración.

Criterios de éxito (fijados el 2026-11-02, antes de evaluar):

1. F1 a nivel de categoría CIE-10 (3 caracteres) ⩾ 0,50 en 50 casos de dev.
2. Cero diagnósticos negados sugeridos como confirmados en las 15 notas ficticias.
3. Tiempo medio por nota ⩽ 120 s con `qwen2.5:7b-instruct` en local.

Es un prototipo académico, no una herramienta clínica. Solo se usaron el corpus público CodiEsp y notas ficticias; no se usó ninguna historia clínica real.

2. Datos

Se usó el corpus público CodiEsp (CLEF eHealth 2020), subtarea de diagnósticos (CodiEsp-D), y un catálogo CIE-10 de 71 486 códigos con descripción en español e inglés.

Conjunto	Casos	Uso en el proyecto
train	500 (con anotaciones)	Construir el catálogo, medir cobertura y ajustar reglas
dev	250 (con anotaciones)	Evaluación: 50 casos fijos, semilla 42
test	3001 textos (sin anotaciones)	No usado: no tiene códigos de referencia
Notas ficticias	15 (9 con negación)	Demo y prueba de exclusión de negados

Hallazgos principales del análisis exploratorio:

- Las notas son largas: media de 349 palabras en train y 352 en dev.
- Cada caso tiene muchos códigos: media de 11,32 códigos de diagnóstico por caso en train y 10,73 en dev. En train + dev hay 2194 códigos distintos, con una cola larga de códigos poco frecuentes.
- Los códigos más frecuentes son síntomas: R52, R69, R50.9, I10 y R59.9; 15 de los 30 más frecuentes pertenecen al capítulo R.
- Abreviaturas frecuentes: TAC (1213 ocurrencias), PCR (506), VIH (331), HTA (95). Algunas, como TA, FC o FR, son ambiguas.
- CodiEsp no etiqueta la negación, por lo que la exclusión de negados solo puede medirse con las notas ficticias.

Cobertura del catálogo. 182 de 1767 códigos de train (10,3 %) y 95 de 1158 de dev (8,2 %) no existen en el catálogo. Todos son códigos padre cuyo subcódigo sí existe (por ejemplo, el gold C34.9 frente a C34.90 en el catálogo). En los 50 casos de evaluación, 13 (26 %) tienen algún código de referencia fuera del catálogo, lo que fija un techo para cualquier sistema que solo emita códigos válidos.

CodiEsp usa CIE-10-ES (versión española, derivada de CIE-10-CM, más granular), mientras que Ecuador usa la CIE-10 de la OMS. Por eso los resultados se reportan también a nivel de categoría de 3 caracteres, que es comparable entre ambas versiones.

3. Desarrollo por fases CRISP-ML(Q)

El proyecto recorrió las seis fases; cada una quedó registrada en `docs/BITACORA.md` con sus cifras y las rutas de los archivos de `results/` que las respaldan.

Fase 1 — Comprensión del negocio y de los datos. Se definió el problema, el objetivo y los tres criterios de éxito antes de evaluar. Un análisis exploratorio (`scripts/eda.py`) midió casos por partición, longitud, códigos por caso, frecuencia de abreviaturas y de expresiones de negación. Se identificaron los riesgos principales: CodiEsp sin etiqueta de negación, cola larga de códigos y posible desajuste de versiones del catálogo.

Fase 2 — Preparación de datos. Se normalizaron los códigos a mayúsculas y sin punto (0 colisiones; reversibilidad garantizada con la columna del código original), se generó el catálogo en Parquet (1910 categorías), se midió la cobertura frente a CodiEsp, se seleccionaron 50 casos de dev con semilla 42 y se construyó un diccionario de 87 abreviaturas, incluido vocabulario ecuatoriano (pcte, APP, EGO, IESS). Se crearon 15 notas ficticias validadas: todos sus códigos existen en el catálogo y todas sus justificaciones son texto literal.

Fase 3 — Ingeniería del modelo. Se construyeron cuatro componentes:

1. Preprocesamiento: normalización, expansión de abreviaturas con mapa de offsets (la evidencia siempre se recupera del texto original) y segmentación en oraciones.
2. Detección de negación por reglas tipo NegEx/ConText en español, con estados afirmado, negado y sospecha.
3. Recuperación RAG: embeddings `intfloat/multilingual-e5-base` sobre las 71 486 descripciones del catálogo, con búsqueda exacta en NumPy y caché en disco.
4. Pipeline con LLM intercambiable (Ollama o Gemini, elegido con `LLM_PROVIDER` en `.env`), salida JSON validada con pydantic y validación final de código y evidencia.

Fase 4 — Evaluación. Se evaluó la recuperación de forma completa (50 casos, 521 códigos de referencia) y se probaron variantes: descripciones en inglés, combinación español-inglés, expansión de abreviaturas y distintos k. El pipeline completo se evaluó de forma parcial con `scripts/evaluate.py` (precisión, exhaustividad y F1 micro y macro, a código completo y a categoría), con una ablación sin módulo de negación.

Fase 5 — Despliegue. Se construyó una aplicación Streamlit donde se ingresa la nota, se elige el proveedor y se ven los códigos sugeridos con la evidencia resaltada (verde = confirmado, gris tachado = negado). También existe una CLI: `python -m codificamed "nota"` .

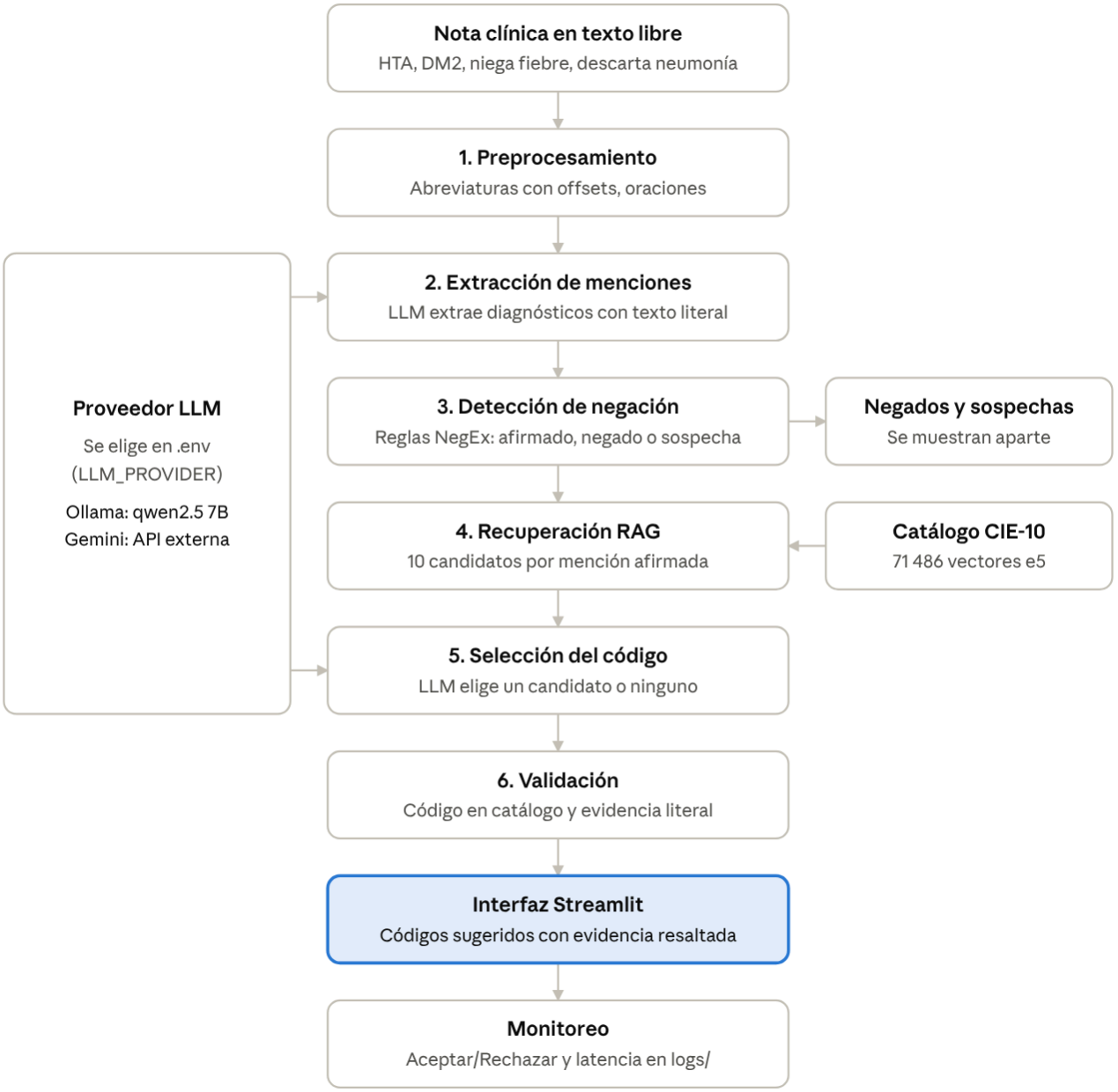
Fase 6 — Monitoreo y mantenimiento. La aplicación registra las decisiones Aceptar/Rechazar del codificador en `logs/feedback.csv` (sin el texto de la nota) y la latencia y errores en `logs/latencia.csv` . `scripts/monitor.py` resume la tasa de

aceptación por proveedor y código y emite alertas: aceptación menor a 0,60, latencia mayor a 120 s o errores por encima del 10 %. Se documentó cómo actualizar el catálogo y las abreviaturas, y cuándo reevaluar.

4. Arquitectura de la solución

La solución es un pipeline híbrido: reglas para limpiar el texto y detectar negación, recuperación semántica (RAG) sobre el catálogo CIE-10 para proponer candidatos, y un LLM intercambiable que extrae los diagnósticos y elige el código.

El LLM solo actúa en la extracción y en la selección del código



arquitectura de CodificaMed · 6 pasos, LLM intercambiable

El proveedor LLM solo interviene en los pasos 2 y 5. El preprocesamiento, la negación, la recuperación y la validación son idénticos para ambos proveedores, de modo que la comparación local frente a API mide el efecto del modelo y nada más.

Módulo	Función
`preprocess.py`	Normalización, expansión de abreviaturas con mapa de offsets, segmentación
`negation.py`	Reglas NegEx/ConText en español: afirmado, negado, sospecha
`retrieval.py`	Embeddings multilingual-e5-base, caché en disco, búsqueda exacta en NumPy
`llm/`	Interfaz común y proveedores Ollama y Gemini; temperatura 0 y reintentos
`schemas.py`	Esquemas pydantic de las respuestas del LLM y de la salida
`pipeline.py`	Orquesta los seis pasos y valida código y evidencia
`feedback.py`	Registro de decisiones y latencia en `logs/`
`app/streamlit_app.py`	Interfaz para ingresar la nota y ver los códigos con su evidencia
`scripts/`	Preparación de datos, índice, evaluación, comparación y monitoreo

Tecnologías de PLN aplicadas y su justificación: RAG con embeddings, porque el catálogo tiene 71 486 códigos y no cabe en el contexto de un LLM; detección de negación por reglas, porque es un requisito explícito y las reglas son explicables; y extracción de entidades clínicas con LLM, porque maneja abreviaturas y errores de tipeo sin entrenar un modelo propio. Stack: Python 3.12, uv, sentence-transformers, pydantic, Ollama, google-genai, Streamlit y pytest.

5. Resultados de la evaluación

La recuperación se evaluó completa y limita el desempeño: incluso con menciones perfectas, el código de categoría correcto está entre los 10 candidatos en el 65,3 % de los casos. El pipeline completo solo se evaluó sobre una muestra parcial, así que sus cifras son indicativas.

5.1 Detección de negación

Las reglas acertaron 44 de 44 frases de prueba (precisión y exhaustividad de 1,0 en la clase negado) y 27 de 27 menciones en las notas ficticias. Estas cifras no estiman el desempeño real: las frases se escribieron junto con las reglas y CodiEsp no tiene etiquetas de negación.

5.2 Recuperación de candidatos

Medida sobre 50 casos de dev, 521 códigos de referencia y 589 menciones oráculo (las anotadas en CodiEsp), con descripciones en español:

Criterio	recall@1	recall@5	recall@10	recall@20	recall@30
Código exacto	0,167	0,288	0,374	0,472	0,530
Categoría (3 caracteres)	0,370	0,557	0,653	0,749	0,797

Ni las descripciones en inglés (0,324 exacto@10), ni la combinación español-inglés (0,380), ni la expansión de abreviaturas mejoraron el recall@10 en al menos 0,01, por lo que se mantuvo la variante en español. Aumentar k sí sube el recall, a costa de dar al LLM más candidatos entre los que elegir. Los fallos típicos son menciones vagas (“enfermedad”, “adenopatías”), diferencias de especificidad (“infarto agudo de miocardio”) y confusiones anatómicas.

5.3 Pipeline completo: modelo local frente a API externa

La evaluación prevista era de 50 casos de dev y 15 notas ficticias por proveedor. Por tiempo y capacidad de cómputo (CPU sin GPU) se interrumpió y solo se procesaron las notas que se indican:

Métrica	Gemini (gemini-3.5-flash-lite, API)	Ollama (qwen2.5:7b-instruct, local)
Casos dev evaluados	6	0
Notas ficticias evaluadas	6	5
F1 micro, categoría (dev)	0,348 (precisión 0,696; exhaustividad 0,232)	No evaluado
F1 micro, código completo (dev)	0,265 (precisión 0,542; exhaustividad 0,176)	No evaluado
F1 macro, categoría (dev)	0,327	No evaluado
F1 micro (ficticias)	0,375	0,100

Métrica	Gemini (gemini-3.5-flash-lite, API)	Ollama (qwen2.5:7b-instruct, local)
Tiempo medio por nota (ficticias)	11,0 s	168,0 s
Tiempo medio por nota (dev)	57,2 s	No evaluado
Negados sugeridos como confirmados	0 de 4	0 de 4
Costo	0 USD (cuota gratuita)	0 USD (local)

En la prueba manual con la misma nota ficticia, ambos proveedores codificaron “cefalea occipital” como R51 con evidencia literal y separaron “dolor torácico” como negado; Gemini además codificó DM2 como E11.9, mientras que el modelo local no extrajo HTA ni DM2. La diferencia principal entre proveedores está en la extracción de menciones y en el tiempo.

Ablación sin negación (Gemini, 6 notas ficticias): 0 de 4 negados sugeridos como confirmados, igual que con negación, y F1 micro de 0,353 frente a 0,375. En esta muestra el propio LLM no extrajo los negados como diagnósticos, así que no se puede concluir el aporte aislado del módulo de reglas.

5.4 Cumplimiento de los criterios de éxito

Criterio	Resultado medido	Estado
F1 categoría ≥ 0,50 en 50 casos de dev	0,348 en 6 casos (Gemini)	No alcanzado en la muestra; sin medir en 50 casos
0 negados sugeridos como confirmados en 15 ficticias	0 de 4 en 6 notas (Gemini) y en 5 notas (Ollama)	Sin incumplimientos en lo evaluado; faltan notas
Tiempo medio ≤ 120 s con qwen2.5:7b local	168,0 s en 5 notas ficticias	No alcanzado

6. Conclusiones

1. **El prototipo cumple la funcionalidad pedida.** Recibe una nota en texto libre, sugiere códigos CIE-10 con la frase literal que los justifica, separa los diagnósticos negados y cambia entre modelo local y API editando solo `.env`. Por diseño, nunca emite un código fuera del catálogo ni una evidencia que no esté en la nota.

2. **La exclusión de negados funcionó en todo lo evaluado.** Ningún diagnóstico negado se sugirió como confirmado con ninguno de los dos proveedores, aunque la muestra es pequeña.
3. **La recuperación es el cuello de botella.** Con menciones perfectas, el recall de categoría@10 es 0,653; el LLM no puede elegir un código que no recibe. Cambiar el idioma de las descripciones o expandir abreviaturas no ayudó.
4. **La precisión es aceptable, la exhaustividad no.** Con Gemini, 7 de cada 10 categorías sugeridas son correctas (precisión 0,696), pero solo se recupera cerca de un cuarto de las de referencia (0,232). Para el codificador esto significa sugerencias confiables pero incompletas.
5. **La API externa superó al modelo local en este equipo.** Gemini fue unas 15 veces más rápido en notas ficticias (11 s frente a 168 s) y extrajo más diagnósticos (F1 0,375 frente a 0,100). El modelo local tiene la ventaja de no enviar datos fuera del equipo, pero en CPU no cumple el criterio de tiempo.
6. **Los criterios de F1 y tiempo no se alcanzaron en la muestra evaluada**, y la evaluación completa queda pendiente. La metodología CRISP-ML(Q) permitió detectar estos límites con cifras trazables y dejó claro dónde mejorar.

7. Limitaciones y trabajo futuro

Limitaciones

- **Evaluación parcial del pipeline:** 6 casos de dev y 5 a 6 notas ficticias en lugar de 50 y 15; no hay F1 del modelo local sobre dev, por lo que la comparación local frente a API es incompleta.
- **Sesgo de selección:** la variante de recuperación se eligió sobre los mismos 50 casos de dev previstos para la evaluación final (sesgo pequeño: se mantuvo la variante de partida).
- **Métricas de recuperación con menciones oráculo:** con menciones extraídas por el LLM el recall real es menor.
- **Negación sin referencia real:** CodiEsp no etiqueta negación y las frases de prueba se escribieron junto con las reglas.
- **Techo por catálogo:** 26 % de los casos de evaluación tiene códigos padre que no existen en el catálogo.
- **Versión de CIE-10:** CodiEsp usa CIE-10-ES; Ecuador usa la CIE-10 de la OMS.
- **Sin validación clínica** de abreviaturas, reglas de negación ni notas ficticias.
- **Privacidad:** la cuota gratuita de Gemini puede usar los datos enviados; por eso solo se enviaron CodiEsp y notas ficticias.
- **Monitoreo sin datos reales:** los umbrales de alerta son propuestos, no calibrados.

Trabajo futuro

- Completar la evaluación de 50 casos y 15 notas con ambos proveedores (`scripts/evaluate.py` es reanudable), idealmente con GPU.
- Mejorar la recuperación con búsqueda híbrida léxica-semántica, contexto de la oración y reranking con el LLM.
- Mejorar el prompt de extracción del modelo local o probar un modelo más ligero para cumplir el tiempo.
- Construir un conjunto de negación anotado a mano y obtener revisión clínica.
- Adaptar el catálogo a la CIE-10 de la OMS usada en Ecuador.